People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research

Laboratoire STIC

Mobile Operating Systems

Session 3: General Architecture of Mobile Operating Systems

Dr. Abdessamad SAIDI

University Abou Bekr Belkaid of Tlemcen
Faculty of Sciences
Computer Science Department

22nd, February, 2026

Opening Question

When you tap an app icon, what happens inside the OS?

- Does the app talk directly to hardware?
- Who decides memory allocation?
- Where is security enforced?

Session Objectives

By the end of this session, you will be able to:

- Understand the generic layered architecture of mobile OS
- Identify kernel, middleware, and application layers
- Compare mobile OS architecture with classical OS
- Explain why layering is essential for security and performance

Why Do We Need an Architecture?

Architecture answers:

- Who does what?
- Who can access what?
- How complexity is managed?
- How the system evolves safely?

Architecture as a Contract

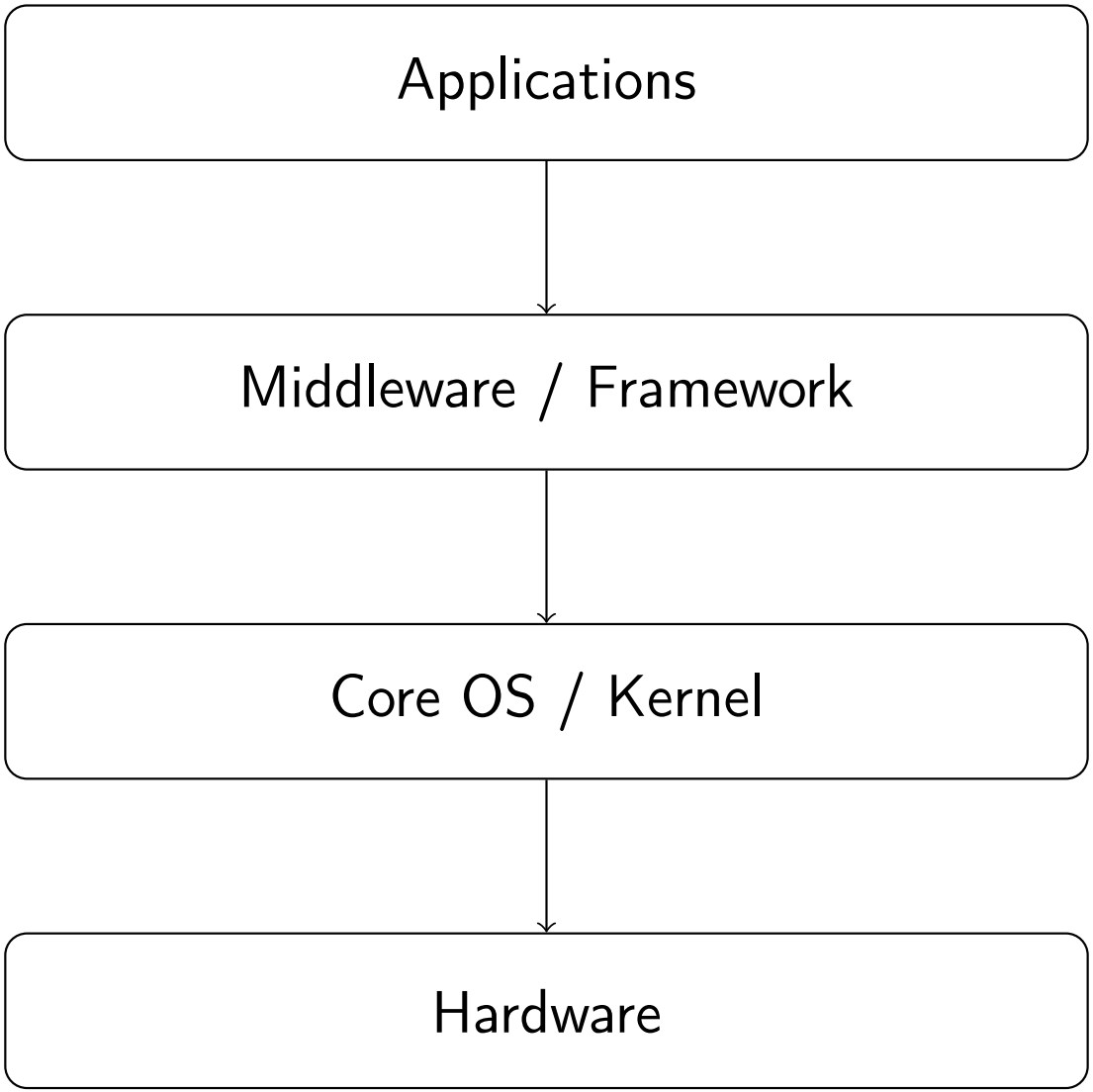
- Defines clear responsibilities
- Limits coupling between components
- Protects system integrity
- Enables parallel development

The Generic Mobile OS Architecture

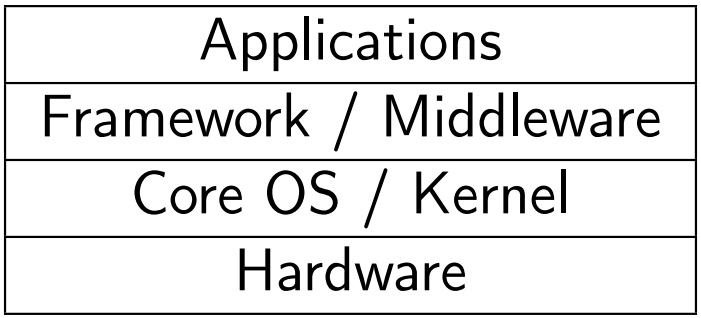
Most mobile OS follow a layered model:

1. Applications
2. Middleware / Framework
3. Core OS / Kernel

Generic Mobile OS Layered Architecture



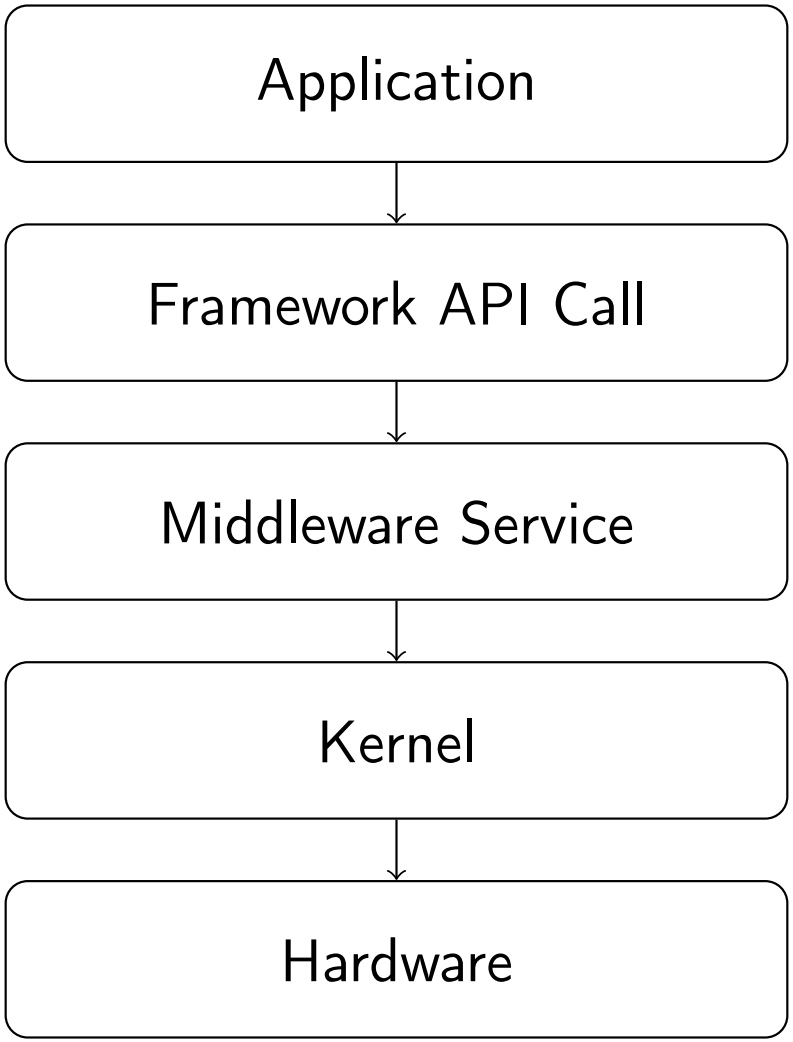
High-Level View



Application Layer — Role

- Contains user-facing applications
- No direct hardware access
- Runs in sandboxed environments
- Uses OS-provided APIs

How an Application Executes (Simplified Flow)



Why Applications Are Isolated

- Prevent malicious behavior
- Avoid crashes propagating
- Protect user data

Pedagogical Question

Why shouldn't apps access hardware directly?

Middleware — Definition

Middleware is the software layer that:

- Exposes system services to apps
- Hides hardware complexity
- Enforces security and policies

Typical Middleware Services

- UI toolkit
- Resource management
- Location services
- Media services
- Notification system

Middleware as a Policy Enforcer

- Checks permissions
- Controls background execution
- Regulates access to sensors

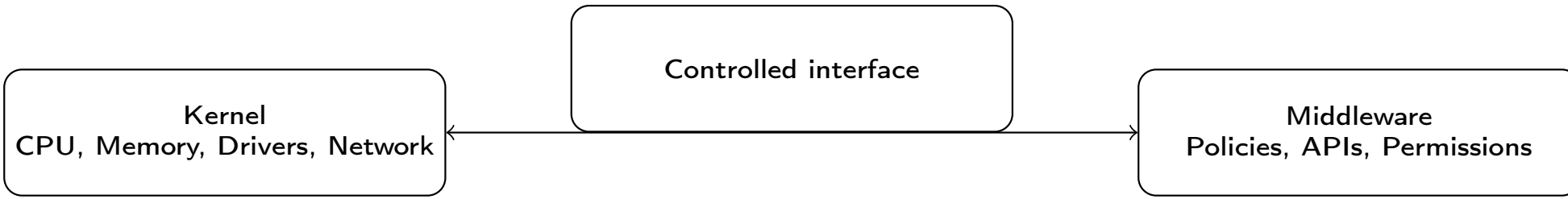
Kernel — Core Responsibilities

- Process management
- Memory management
- File systems
- Networking
- Device drivers

Kernel vs Middleware

- Kernel: low-level, hardware-near
- Middleware: policy and abstraction
- Apps never call kernel directly

Kernel vs Middleware — Conceptual Separation



Energy Management Location

- Kernel handles CPU power states
- Middleware decides when tasks run
- Apps must cooperate

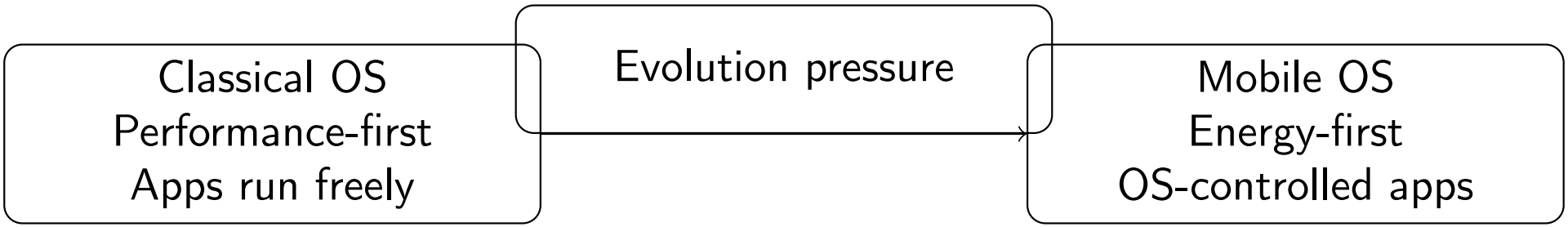
Classical OS Architecture (Reminder)

- Desktop/server OS focus on performance
- Apps often run continuously
- Less aggressive lifecycle control

Key Differences with Mobile OS

- Mobile OS: energy-first design
- Strong app lifecycle control
- Permission-based security
- Touch-centric UI

Classical OS vs Mobile OS — Architectural Emphasis



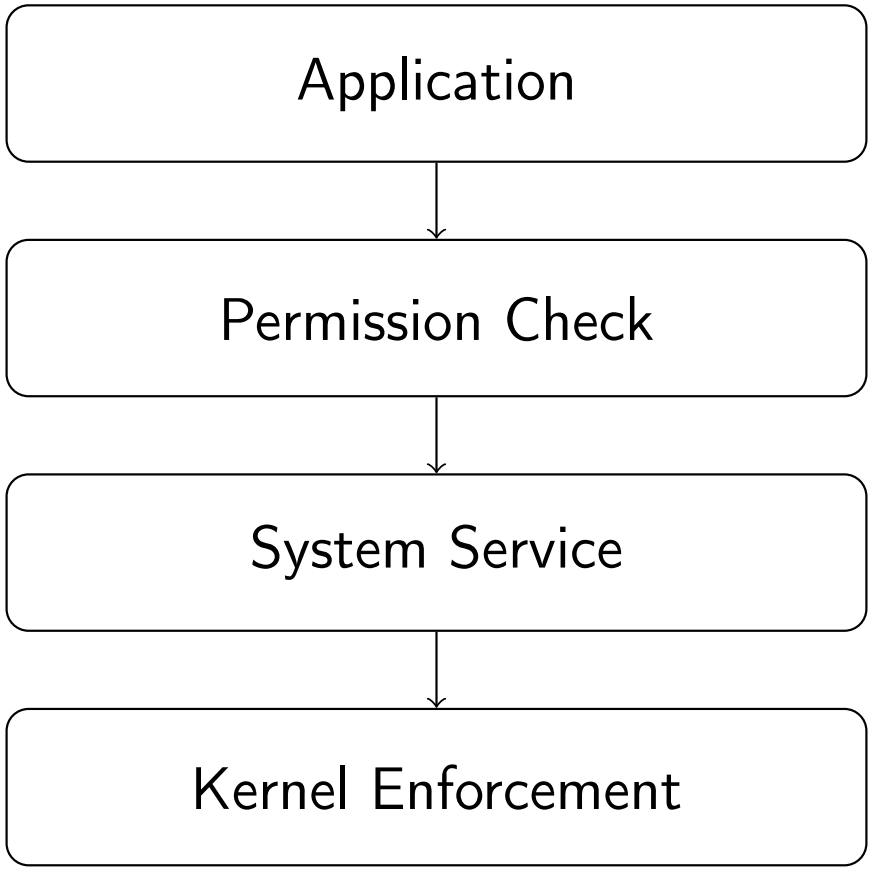
Comparison Table

Aspect	Classical OS	Mobile OS
Energy	Secondary	Primary
App lifetime	Long-running	OS-controlled
Security	User-based	Permission-based
Hardware diversity	Limited	Very high

Security Benefits

- Limits attack surface
- Prevents privilege escalation
- Enables permission enforcement

Security Enforcement Through Layers



Performance Benefits

- Optimized hardware access
- Centralized scheduling
- Reduced redundant work

Maintainability and Evolution

- Easier OS updates
- Backward compatibility
- Vendor customization

Activity: Layer Identification

Question: Where does each belong?

- Camera access API
- Process scheduler
- WhatsApp UI
- File system

Activity — Solution

- Camera API → Middleware
- Scheduler → Kernel
- WhatsApp UI → Application
- File system → Kernel

Key Takeaways

- Mobile OS use layered architecture
- Apps never access hardware directly
- Middleware enforces policy
- Kernel ensures efficiency and stability

Looking Ahead

Next session (Session 4): iOS Overview & History

We will:

- Apply this architecture to iOS
- Explore Apple's design philosophy
- Study multitasking in iOS

End of Session 3

Thank you — see you next Sunday